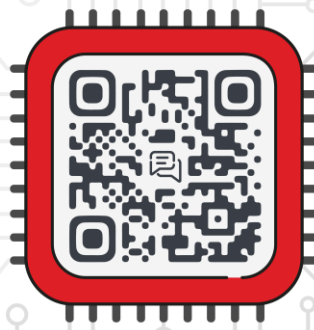




7



14



1680 sp

نظري

كلية الهندسة المعلوماتية

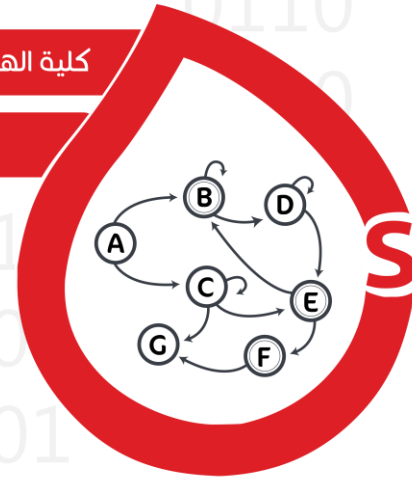
السنة الثالثة

Regular Grammer

&

Context Free Grammer

د. هبة حاتم



RB Informatics; 01/06/2025

اللغات الصورية

القواعد المنتظمة Regular Grammer

ذكرنا في المحاضرة الأولى أن اللغات تصنف لأربعة تصنيفات تبدأ بأقل درجة من التعقيد لكن أكثر درجة من الصرامة وهي ال regular languages التي يتم قبولها من قبل ال FSA ، وتحكم هذه اللغة قواعد تسمى Regular Grammer وهي أكثر القواعد صرامة مقارنة بقواعد باقي اللغات في هرمية تشومسكي وهي النوع الثالث في هذه الهرمية.

التعريف الرسمي للقواعد المنتظمة Formal Definition of Regular Grammer

يتم تعريف القواعد المنتظمة عن طريق تركيب مكون من أربعة عناصر: $G = (V, T, S, P)$

- V (variables/non-terminals): مجموعة منتهية من الرموز غير الطرفية non-terminal symbols مثل (A, B, S) والتي تمثل الحالات (states)
- T (terminals): مجموعة منتهية من الرموز الطرفية terminals مثل (a, b, 0,1) وهي أبجدية اللغة (alphabets) التي تولدها القاعدة:
- V و T مجموعتان متباينة أي لا يوجد عناصر مشتركة بينهما $(V \cap T = \emptyset)$
- S (start symbol): رمز مخصص non-terminal ينتمي V و $(S \in V)$ وهو نقطة البداية لتوليد السلاسل المحرفية strings
- P (production rules): مجموعة منتهية من قواعد الإنتاج من الشكل

حيث:

- α و β هم strings من المجموعة $V \cup T$ (أي هم سلاسل مكونة من مجموعة من ال terminals وال non-terminals)
- α يجب أن تحوي على الأقل رمز non-terminal واحد من V



- قواعد الإنتاج تحدد كيف نقوم باستبدال الرموز non-terminal برموز أخرى سواء كانت non-terminals or terminals
- قواعد اللغة مصنفة إلى أنواع (regular, context free, context sensitive and unrestricted) بناءً على درجة القيود التي تفرضها قواعد الإنتاج P
- القواعد المنتظمة هي مجموعة جزئية من القواعد الأعلى منها حيث الإنتاج فيها محدد بأن يكون إما left linear أو right linear

Left-Linear Grammer

(أي يكون ال non-terminal إلى يسار ال terminal)
قواعد الإنتاج تكون من الشكل $A \rightarrow a$ أو $A \rightarrow Ba$

Right-Linear Grammer

(أي يكون ال non-terminal إلى يمين ال terminal)
قواعد الإنتاج فيها من الشكل $A \rightarrow aB$ أو $A \rightarrow a$
حيث A و non-terminals B وال terminal a ويمكن أن يكون ϵ

لا يُسمح بالدمج بين ال Right-Linear وال Left-Linear في القواعد المنتظمة، يجب ان تكون إما Right أو Left ولكن ليس معا، وذلك لأن الدمج بينهما يولد أنماط تتطلب وجود تخزين لتمثيلها ونحن نعلم أن FSA ال هي memoryless اي لا تملك ذاكرة أو ذاكرتها صغيرة جداً.
وهذه واحدة من القواعد الصارمة التي تحكم اللغات المنتظمة.

مثال 1:

لدينا لغة $L = \{ w \mid w \text{ has an even number of } a's \}$ تقبل السلاسل التي تحوي على عدد زوجي من a
ال Production rules لها :

$$S \rightarrow aA \mid \epsilon$$

$$A \rightarrow aS$$

للتحقق:

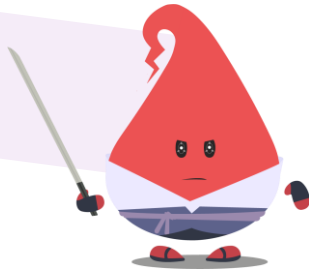
(ملاحظة نقوم بالتحقق هنا للتدريب على كيفية التعامل مع قواعد الإنتاج وليس للإثبات لأن الأمر مثبت من نص السؤال)

بداية سنوضح التعريف الرسمي لقاعدة اللغة formal definition

$$G = (\{ S, A \}, \{ a \}, S, \{ S \rightarrow aA \mid \epsilon, A \rightarrow aS \})$$

لدينا states يجب ان نقوم باستبدالها بحسب قواعد الإنتاج على التالي rule by rule لنولد مجموعة ال strings التي تنتمي للغة التي تولدها هذه القاعدة
نبدأ بالحالة الابتدائية S:

$$S \rightarrow aA$$



$$\begin{aligned} S &\rightarrow aaS \\ S &\rightarrow aaaA \\ S &\rightarrow aaaaS \end{aligned}$$

هنا عوضنا $S = \varepsilon$ $S \rightarrow aaaa$
وكلمنا عوضنا أكثر نلاحظ أن عدد a زوجي دائماً

ملاحظة مهمة:

اللغة التي تولدها ال regular grammer يجب أن تحوي على terminals فقط لذلك يجب أن نقوم باستبدال جميع ال non-terminals حسب قواعد الإنتاج.

مثال 2:

$$G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$$

لتوليد السلسلة ab :

- نبدأ بالحالة الابتدائية S ونطبق القاعدة 1 ($S \rightarrow AB$)

$$S \rightarrow AB$$

- نطبق القاعدة 2 ($A \rightarrow a$)

$$AB \rightarrow aB$$

- نطبق القاعدة 3 ($B \rightarrow b$)

$$aB \rightarrow ab$$

لا يمكن أن نطبق أي قواعد إضافية هنا بالتالي اللغة التي تولدها هذه القاعدة $L(G)$ هي $\{ab\}$

التكافؤ مع ال FSA – Equivalence to Finite Automata

كل regular grammar يمكن تحويلها إلى NFA و العكس صحيح.

الرموز non-terminal تكافئ الحالة state في الاوتوماتا.

الرموز terminal توافق الانتقال.

مثال: قاعدة الإنتاج $S \rightarrow aA$ تكافئ $S \xrightarrow{a} A$

القواعد المنتظمة لا تستطيع التعبير عن (توليد) لغات غير منتظمة مثل $\{a^n b^n \mid n \geq 0\}$ وإنما بحاجة للقواعد الحرة من السياق

Limitations of Regular Grammar

القواعد المنتظمة تقتصر على اللغات التي لا تتطلب:

1. البنى المتداخلة Nested structures ؛ مثل مسألة الاقواس المتوازنة balanced parentheses

2. العد غير المحدود Unbounded counting, مثل $a^n b^n$ بحاجة لتخزين n لتوليد b بنفس العدد

تطبيقات القواعد المنتظمة

هي أساسية في:

- Lexical analysis التحليل المعجمي (تقسيم الجمل ل tokens في المترجمات compilers)
- pattern matching مطابقة الأنماط
- text processing معالجة النصوص مثل محركات البحث

اشتقاقات من القاعدة Derivations from Grammar

اللغة $L(G)$ المولدة من القاعدة G هي مجموعة جميع السلاسل المحرفية التي يمكن اشتقاقها (استنتاجها أو التوصل إليها) ابتداءً من الحالة الابتدائية S

مثال 1:

لدينا القاعدة $G1 = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\})$

نبدأ بالحالة الابتدائية دائماً:

- $S \Rightarrow aAb$ [Apply Rule 1]
- $aAb \Rightarrow aaAbb$ [Apply Rule 2 on $aA \rightarrow aaAb$]
- $aaAbb \Rightarrow aaaAbbb$ [Apply Rule 2 again]
- $aaaAbbb \Rightarrow aaabbbb$ [Apply Rule 3 ($A \rightarrow \epsilon$)]

نلاحظ ان اللغة التي تولدها هي جميع السلاسل المحرفية التي تحوي نفس العدد غير الصفري من a و b

$$L(G1) = \{a^n b^n \mid n > 0\}$$

مثال 2:

لدينا القاعدة $G2 = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow aA \mid a, B \rightarrow bB \mid b\})$

عموماً هناك طريقتان للحل، إما التجريب اعتباطياً أو السير بترتيب منهجي معين.
حالياً سنتبع الطريقة الاعتباطية وسنتحدث عن الطريقة المنهجية بالتفصيل لاحقاً.

1. توليد السلسلة ab :

$$S \Rightarrow AB \Rightarrow aB \Rightarrow ab$$

2. String aab :

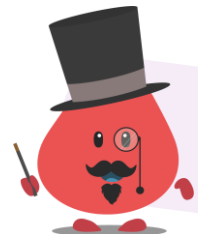
$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aab$$

(Using $A \rightarrow aA$, then $A \rightarrow a, B \rightarrow b$)

3. String $aabb$:

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb$$

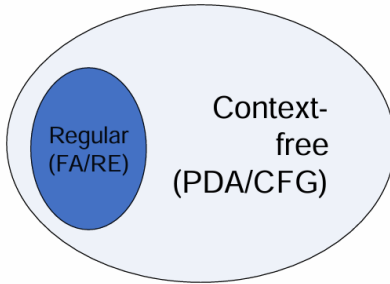
(Using $A \rightarrow aA, B \rightarrow bB$, then $A \rightarrow a, B \rightarrow b$)



اللغة المولدة هي جميع السلاسل التي تحوي على a واحد أو أكثر متبوع ب b واحد أو أكثر
 $L(G_2) = \{ a^m b^n \mid m \geq 1 \text{ and } n \geq 1 \}$

Context-Free Languages اللغات الحرة من السياق

اللغات الحرة من السياق هي النوع الثاني و هي أعم من اللغات المنتظمة في هرمية تشومسكي ويتم توليدها عن طريق القواعد الحرة من السياق context-free grammar و يتم تمثيلها عن طريق ال Pushdown Automata .



ومن تطبيقاتها:

- أشجار التحليل parse trees
- المترجمات Compilers
- لغة XML

التعريف الرسمي للقواعد الحرة من السياق Formal Definition of CFG:

يتم تعريف ال CFG عن طريق تركيب مكون من أربعة عناصر: $G = (V, \Sigma, S, P)$

- V : مجموعة من الرموز غير الطرفية non-terminal symbols
- Σ : مجموعة من الرموز الطرفية (alphabet) terminals
- S (start symbol) $(S \in V)$
- P (production rules): قواعد الإنتاج من الشكل $A \rightarrow \alpha$

مثال:

لتوليد اللغة $a^n b^n$ تكون ال CFG: $G = (\{S, A\}, \{a, b\}, \{S \rightarrow aAb, A \rightarrow aAb \mid \epsilon\})$

لنقم باشتقاق $a^3 b^3$

1. Start with S :

$$S \Rightarrow aAb \quad (\text{using Rule 1})$$

2. Expand A :

$$aAb \Rightarrow aaAbb \quad (\text{using } A \rightarrow aAb)$$

3. Expand A again:

$$aaAbb \Rightarrow aaaAbbb \quad (\text{using } A \rightarrow aAb)$$

4. Terminate A :

$$aaaAbbb \Rightarrow aaabbb \quad (\text{using } A \rightarrow \epsilon)$$



وهي جميع السلاسل المحرفية التي تحوي نفس العدد غير الصفري من a و b

$$L(G) = \{ a^n b^n \mid n > 0 \}$$

ملاحظة:

غالباً نستخدم التعويض ب ε في النهاية للتوصل للنمط النهائي

Palindromes

هي الكلمة التي تكون نفسها عند قراءتها من اليمين لليساو أو من اليسار إلى اليمين، مثل:

Madam , Redivider , Malayalam, 010010010

ال CFG الخاص بها: $G = (\{A\}, \{0,1\}, P, A)$

$P(\text{production rules}): A \Rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \varepsilon$

نلاحظ أنها right-linear و left-linear معاً بذلك هي ليست منتظمة.

ملاحظات

- ال string المدخل يكون مقبول إذا و فقط إذا كان بإمكان ال CFG أن تولده
- لا يهم ترتيب العناصر في ال formal definition , فهنا مثلاً نلاحظ أن الحالة الابتدائية تم ذكرها بعد قواعد الإنتاج.
- الرموز ليست ثابتة بالضرورة فكما لاحظنا أن مجموعة ال non-terminals تم ترميزها ب Σ و V

قاعدة تفكير لتوليد string من ال grammar

- نختار تسلسل تنفيذ قواعد الإنتاج بالشكل الذي سيولد ال string
- بكل خطوة نقوم باستبدال ال non-terminal بواحدة من القواعد ال productions الخاصة به

مثال:

$w = 01110$

يمكن ل G أن تولد w كالآتي:

1. $A \Rightarrow 0A0$
2. $\Rightarrow 01A10$
3. $\Rightarrow 01110$



أمثلة أخرى عن استخدامات ال CFL

مطابقة الأقواس (الأقواس المتوازنة) في الكود البرمجي

Syntax checking

في السيناريوهات التي تحتاج إلى:

مطابقة رمز مع رمز آخر

مطابقة عدد تكرار رمز مع عدد تكرار رمز آخر

استبدال رمز ما بشكل متكرر بسلسلة من الرموز الأخرى.

مثال على النقطة الثالثة:

لنفترض أن لدينا قاعدة الاستبدال $A \rightarrow AB, B \rightarrow A$ وبدأنا بالرمز A :

التكرار:

1. A

2. $A \rightarrow AB$

3. $AB \rightarrow ABA$ (لأن A صارت AB و AB صارت A)

4. $ABA \rightarrow ABAAB$

$ABAAB \rightarrow ABAABABA$

...وهكذا بشكل تكراري.

لغة الأقواس المتوازنة (balanced parenthesis)

$$G = (\{S\}, \{ (,) \}, S, P)$$

$$P: S \Rightarrow (S) \mid SS \mid \varepsilon$$

(الalphabet هنا هم الأقواس "(" و ")")

لتقبل اللغة string المدخل يجب أن يحتوي على عدد متساوي من ال (و) و كل قوس مفتوح له قوس مغلق و بالترتيب الصحيح, مثال عليها:

$() , (()) , ()() , (())()$

$$S \rightarrow (S) \rightarrow (SS) \rightarrow (SSS) \rightarrow ((S)(S)(S)) \rightarrow (((S))()) \rightarrow (((()))())$$

القاعدة من أجل $L = \{0^m 1^n \mid m \geq n\}$

$$G = (\{S, A\}, \{0, 1\}, S, P)$$

$$P: S \rightarrow 0S1 \mid A$$

$$A \rightarrow 0A \mid \varepsilon$$

القاعدة تقوم بتوليد strings حيث عدد الأصفار فيها على الأقل نفس عدد الواحدات.

استراتيجية الحل:

1. نستخدم القاعدة $S \Rightarrow 0S1$ لتوليد عدد متساوي من الأصفار و الواحدات

2. نستخدم القاعدة $A \Rightarrow 0A$ لإضاف أصفار إضافية



اشتقاق 00000111 ($m=5$, $n=3$)**Step 1:** Generate 3 balanced 0-1 pairs using $S \rightarrow 0S1$:**Step 2:** Switch to A to add extra 0 s: $000S111 \Rightarrow 000A111$ (Rule 2: $S \rightarrow A$)**Step 3:** Use $A \rightarrow 0A$ twice to add 2 extra 0 s:**Step 4:** Terminate with $A \rightarrow \epsilon$: $00000A111 \Rightarrow 00000111$ (Rule 4)**Derivation Summary** $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000A111 \Rightarrow 0000A111 \Rightarrow 00000A111 \Rightarrow 00000111$ More examples

- $L1 = \{0^n \mid n \geq 0\}$
- $L2 = \{0^n \mid n \geq 1\}$
- $L3 = \{0^i 1^j 2^k \mid i = j \text{ or } j = k, \text{ where } i, j, k \geq 0\}$
- $L4 = \{0^i 1^j 2^k \mid i = j \text{ or } i = k, \text{ where } i, j, k \geq 1\}$

تطبيقات ال CFLs وال CFGs

الترجمات compilers تستخدم ال المحلّلات parsers (وهي مكون برمجي software component) لتحليل الكود و تدقيقه قواعديا

يمكن التعبير عن المحلّلات باستخدام ال CFGs مثل :

▪ Balancing parenthesis

$$B \rightarrow BB \mid (B) \mid \text{Statement}$$

$$\text{Statement} \rightarrow \dots$$

▪ If-then-else

$$S \Rightarrow SS \mid \text{if Condition then Statement else Statement} \mid \text{if Condition then Statement} \mid \text{Statement}$$

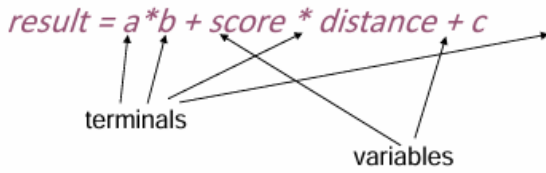
$$\text{Condition} \rightarrow \dots$$

$$\text{Statement} \rightarrow \dots$$

▪ parenthesis matching { ... } مطابقة الأقواس

لا تكتفي بالتحقق من التوازن، بل تتبع كل قوس مفتوح وتحدد القوس المغلق المطابق له.

Syntactic Expressions in Programming Languages التعبيرات النحوية في لغات البرمجة



العمليات الحسابية هي terminals أيضاً
اللغات المنتظمة تحوي على terminals فقط.

إذا اعتبرنا اللغة تحوي على الأحرف a و b وتحتوي 0 و 1 فيمكن كتابة regular expression كالتالي:
Regular expression = $(a + b)(a + b + 0 + 1)^*$

ملاحظة

ال * في ال regular expressions تعني ال kleane star اما في ال CFL فهي عملية الضرب.

طرق لتحديد إذا كان String ينتمي لقاعدة محددة

■ نبدأ بالحالة الابتدائية للقاعدة

ونقوم بتعويض ال (PR) Production rule الخاصة بها التي تماثل او قريبة بشكل كافي لبنية ال string الذي نسعى للوصول إليه

■ نقوم بتعويض ال PRs بشكل تسلسلي

نقوم باستبدال كل متغير non-terminal في ال string الحالي الذي وصلنا إليه بال PR الأنسب للوصول لل string النهائي
نكرر هذه العملية حتى:

- نصل لل string المطلوب (success)
- أو لايمكن اشتقاق المزيد من ال string و نحن لم نصل بعد للمطلوب (failure)

- هذه الطريقة هي تحليل من الأعلى للأسفل أي نبدأ من البداية (الحالة الابتدائية) لنصل للنهاية (ال string المطلوب)
- بشكل عام التعويض الذي نقوم به ليس بشكل عشوائي و إنما تبعاً للقاعدة التي تطابق أو تشابه لحد كبير الشكل النهائي الذي نريده
- إذا وصلنا إلى نقطة معينة و لم نصل للنهاية المطلوبة ومازال هناك PR لم نقم باستخدامها أو ترتيب مختلف لم نجربه نرجع خطوات للوراء و نعيد التجربة (backtracking)

مثال:

w=01110

هل w يمثل palindrome؟

Production Rule:
 $A \Rightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \epsilon$

$A \Rightarrow 0A0$
 $\Rightarrow 01A10$
 $\Rightarrow 01110$



حدد فيما إذا كانت القاعدة :

$$S \rightarrow 0B \mid 1A, A \rightarrow 0 \mid 0S \mid 1AA \mid ^, B \rightarrow 1 \mid 1S \mid 0BB$$

تعطي السلسلة 00110101

- | | |
|---------------------|-----------|
| 1. (S-> 0B) -> 00BB | (B-> 0BB) |
| 2. 001B | (B-> 1) |
| 3. 0011S | (B-> 1S) |
| 4. 00110B | (S->0B) |
| 5. 001101S | (B-> 1S) |
| 6. 0011010B | (S->0B) |
| 7. 00110101 | (B->1) |



ملاحظة

^ هي نفسها ε

حدد فيما إذا كانت القاعدة :

$$S \rightarrow aAb, A \rightarrow aAb \mid ^$$

تعطي السلسلة aabbbb

- S -> aAb
- -> a aAb b (A -> aAb)
- -> a a aAb bb (A -> aAb)
- ✗ • -> aaabbb (A -> ^)

تعميم الاشتقاق Generalization of Derivations

■ $A \Rightarrow X$ (A derives X in a single step) يتم الوصول إلى X من A بخطوة واحدة

■ $A \Rightarrow^*_G X$ (A derives X in a multiple steps) يتم الوصول من X إلى A بعدة خطوات

إذا أمكن الوصول إلى B من A بعدة خطوات، و الوصول إلى C من B بعدة خطوات، بالتالي يمكن الوصول إلى C من A بعدة خطوات

$$\text{IF } A \Rightarrow^*_G B, \text{ and } B \Rightarrow^*_G C, \text{ THEN } A \Rightarrow^*_G C$$

يمكننا صياغة CFG لتقبل التعابير البسيطة:

- $G = (V, T, P, S)$
 - $V = \{E, F\}$
 - $T = \{0, 1, a, b, +, *, (,)\}$
 - $S = \{E\}$
 - P:
 - $E \Rightarrow E+E \mid E * E \mid (E) \mid F$
 - $F \Rightarrow aF \mid bF \mid 0F \mid 1F \mid a \mid b \mid 0 \mid 1$

مثال

قم باشتقاق من G حيث تعطى ال PR كالتالي :

Production :

$$E \Rightarrow E+E \mid E * E \mid (E) \mid F$$

$$F \Rightarrow aF \mid bF \mid 0F \mid 1F \mid \varepsilon$$

الاشتقاق من اليسار :

نقوم بتعويض المتغير في أقصى اليسار في كل خطوة

■ E
 ■ $\Rightarrow E * E$
 ■ $\Rightarrow F * E$
 ■ $\Rightarrow aF * E$
 ■ $\Rightarrow a * E$
 ■ $\Rightarrow a * (E)$
 ■ $\Rightarrow a * (E + E)$
 ■ $\Rightarrow a * (F + E)$
 ■ $\Rightarrow a * (aF + E)$
 ■ $\Rightarrow a * (abF + E)$
 ■ $\Rightarrow a * (ab + E)$
 ■ $\Rightarrow a * (ab + F)$
 ■ $\Rightarrow a * (ab + 1F)$
 ■ $\Rightarrow a * (ab + 10F)$
 ■ $\Rightarrow a * (ab + 10)$

■ E
 ■ $\Rightarrow E * E$
 ■ $\Rightarrow E * (E)$
 ■ $\Rightarrow E * (E + E)$
 ■ $\Rightarrow E * (E + F)$
 ■ $\Rightarrow E * (E + 1F)$
 ■ $\Rightarrow E * (E + 10F)$
 ■ $\Rightarrow E * (E + 10)$
 ■ $\Rightarrow E * (F + 10)$
 ■ $\Rightarrow E * (aF + 10)$
 ■ $\Rightarrow E * (abF + 10)$
 ■ $\Rightarrow E * (ab + 10)$
 ■ $\Rightarrow F * (ab + 10)$
 ■ $\Rightarrow aF * (ab + 10)$
 ■ $\Rightarrow a * (ab + 10)$

الاشتقاق من اليمين :

نقوم بتعويض المتغير في أقصى اليمين في كل خطوة

يمكن أن نعبر عنها ب $E \rightarrow * G a * (ab + 10)$ أي قمنا باشتقاق من E بعدة خطوات.
 نلاحظ ان ترتيب الاستبدال بال PRs اختلف بين الاشتقاق من اليمين و الاشتقاق من اليسار.

مسلمة الضخ للغات الحرة من السياق Pumping lemma for CFL

تستخدم لإثبات أن لغة ليست حرة من السياق

إذا كانت A لغة حرة من السياق CFL، عندها يوجد P(pumping length) طول الضخ الذي يحقق

$|S| > P$ من أجل أي سلسلة S تنتمي ل A حيث يمكن تقسيم S ل 5 أقسام $S = uvxyz$ حيث تحقق:

$$uv^i x y^i z \in A \text{ for every } i \geq 0$$

$$|vy| > 0 \text{ (e. g. } v \text{ and } y \text{ cannot both be empty),}$$

$$|vxy| \leq P$$

مسلمة الضخ تعتمد على طريقة نقض الفرض أي أننا نفرض بالبداية أن اللغة حرة من السياق ثم نقوم بنقض هذا الفرض



خطوات نقض الفرض:

- نفرض ان A حرة من السياق
- يجب ان تملك طول ضخ P (نفرض قيمته عند الحل)
- نختار سلسلة S من A بحيث $|S| > P$
- نقسم S إلى uvxyz
- نثبت أن $uv^i xy^i z \notin A$ من أجل i محددة
- نأخذ بعين الاعتبار جميع الطرق الممكنة لتقسيم S و نثبت أن جميعها ليست حرة من السياق (لا تحقق الشروط الثلاثة معاً)

■ ما معنى ان نأخذ جميع طرق تقسيم S بعين الاعتبار؟
الأمر بشكل أساسي أن نأخذ جميع الطرق الممكنة لتحديد vxy بعين الاعتبار بما أنهم أساس الشروط 2 و 3
لكن أن نختبر جميع التقسيمات الممكنة هو أمر مستحيل لأنها كثيرة جداً لذلك نقوم بالتقسيم تبعاً للنمط الذي تأخذه السلسلة و سنرى توضيح ذلك بالأمثلة.

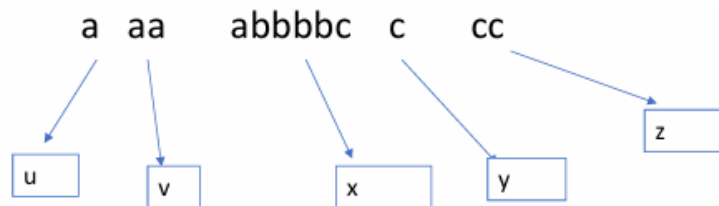
مثال 1:

$$L = \{a^N b^N c^N \mid N > 0\}$$

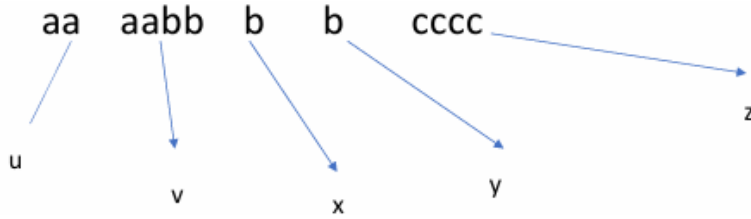
- نفرض في البداية أن L لغة حرة من السياق لها طول ضخ P
 - نختار سلسلة تنتمي ل L : $S = a^P b^P c^P$ حيث $|S| > P$
 - نبدأ الآن بتقسيم السلسلة :
- نقوم باختيار $P = 4$ (يمكن اختيار أي قيمة $1 \leq$)
 $S = aaaaabbbbcccc$
 ذكرنا أنه يجب اختبار جميع طرق تقسيم S الممكنة تبعاً لنمط السلسلة، نلاحظ هنا:

الحالة الأولى:

v و y يحتويان على نوع واحد من الرموز (ليس بالضرورة نفس الرمز ولكن كل منهما يحوي نوع واحد من الرموز فقط)
 بالتالي يمكن تقسيم السلسلة وفقاً لهذه الحالة كالتالي : (يمكن التقسيم بأكثر من طريقة)



لنفرض ان $i=2$ ، إذاً لاختبار الشرط الأول $uv^i xy^i z$ نحصل على $aaaaabbbbcccc$
 وهذا يعني $a^6 b^4 c^5 \notin L$

الحالة الثانية:

إما v أو y يملكان أكثر من نوع من الرموز
يمكن التقسيم كالتالي :

بفرض $i=2$ إذا :

(اختل ترتيب الأحرف بغض النظر عن العدد) $aa aabbaabb b bb cccc \notin L$
و بما أن التقسيم بالحالتين لا ينتمي للغة فاللغة ليست حرة من السياق.

ملاحظة

إذا كانت إحدى حالات التقسيم تنتمي للغة فهذا لا يعني أنها لغة حرة من السياق, فلا إثبات ذلك
يكون عن طريق كتابة CFG او ايجاد PDA.

مثال 2:

ليكن لدينا $L = \{ ww \mid w \in \{0,1\}^* \}$
(أي السلاسل التي يطابق نصفها الأول نصفها الثاني - النصف الأول مكرر مرتين)

- نفرض L لغة حرة من السياق ولها طول ضخم P
 - نختار سلسلة تنتمي ل L ولتكن : $S = 0^P 1^P 0^P 1^P$
 - نبدأ بالتقسيم السلسلة :
- (في المثال السابق لم نأخذ الشرطين 2 و 3 بعين الاعتبار عند التقسيم لكن سنفعل ذلك في هذا المثال)
- نفرض $P = 5$

الحالة الأولى:

vxy ضمن نفس الحد

المقصود بالحدود لهذه السلسلة المختارة :

الحد الأول المنتصف الحد الأخير
00000 11111 00000 11111

يمكن تقسيم السلسلة كالتالي:

00000 11111 00000 11111
u vxy z

نلاحظ أن: $|vxy| = 3 \leq P = 5$ و $|vy| = 2 > 0$ أي الشرطين 2 و 3 محققان

ننتقل للتحقق من الشرط الثالث، بفرض $i=2$, تصبح السلسلة : 00000 111111 00000 11111

أي $0^5 1^7 0^5 1^5$, لا يوجد تطابق بين النصف الأول و الثاني بالتالي السلسلة بهذه التقسيمة لا تنتمي للغة

الحالة الثانية:

vxy يقطع الحد الأول , يمكن التقسيم كالتالي:

00000	11111	00000	11111
u	vxy	z	

نلاحظ أن: $|vxy| = 5 \leq P = 5$ و $|vy| = 4 > 0$ أي الشرطين 2 و 3 محققان

ننتقل للتحقق من الشرط الثالث، بفرض $i=2$ ، تصبح السلسلة : 0000000 111111 00000 11111 أي $0^7 1^7 0^5 1^5$ لا يوجد تطابق بين النصف الأول و الثاني بالتالي السلسلة بهذه التقسيمة لا تنتمي للغة

الحالة الثالثة:

vxy يقطع الحد الأخير , يمكن التقسيم كالتالي:

00000	11111	00000	11111
u	vxy	z	

نلاحظ أن: $|vxy| = 5 \leq P = 5$ و $|vy| = 4 > 0$ أي الشرطين 2 و 3 محققان

ننتقل للتحقق من الشرط الثالث، بفرض $i=2$ ، تصبح السلسلة : 00000 11111 0000000 111111 أي $0^5 1^5 0^7 1^7$ لا يوجد تطابق بين النصف الأول و الثاني بالتالي السلسلة بهذه التقسيمة لا تنتمي للغة

الحالة الرابعة:

vxy يقطع الحد الأوسط , يمكن التقسيم كالتالي:

00000	11111	00000	11111
u	vxy	z	

نلاحظ أن: $|vxy| = 5 \leq P = 5$ و $|vy| = 4 > 0$ أي الشرطين 2 و 3 محققان

ننتقل للتحقق من الشرط الثالث، بفرض $i=2$ ، تصبح السلسلة : 00000 111111 0000000 11111 أي $0^5 1^7 0^7 1^5$ لا يوجد تطابق بين النصف الأول و الثاني بالتالي السلسلة بهذه التقسيمة لا تنتمي للغة

بالتالي بما أن جميع الحالات الممكنة لتقسيم السلسلة لا تنتمي للغة بالتالي اللغة ليست حرة من السياق.

النهاية

"لا تخف أن تتخلى عن الخير
لتذهب للأفضل"

